```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>PumpOS - Master Technical Specification & Architecture</title>
  <style>
    @page {
      size: A4;
      margin: 18mm 15mm 18mm 15mm;
      @bottom-right {
        content: counter(page);
        font-family: 'Helvetica Neue', Arial, sans-serif;
        font-size: 8pt;
        color: #718096;
      }
    }
    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue",
Arial, sans-serif;
      font-size: 9.5pt;
      line-height: 1.45;
      color: #1a202c;
      background-color: #ffffff;
      margin: 0;
      padding: 0;
    }
    .header-block {
      border-bottom: 2.5px solid #0f172a;
```

```
  padding-bottom: 12px;
  margin-bottom: 20px;
}
.header-title {
  font-size: 20pt;
  font-weight: 800;
  color: #0f172a;
  text-transform: uppercase;
  letter-spacing: -0.5px;
  margin: 0 0 4px 0;
}
.header-subtitle {
  font-size: 10pt;
  color: #475569;
  font-weight: 500;
  margin: 0;
}
.meta-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 10px;
  background-color: #f8fafc;
  border: 1px solid #e2e8f0;
  border-radius: 6px;
  padding: 10px 14px;
  margin-bottom: 20px;
  font-size: 8pt;
}
```

```
.meta-item strong {
  display: block;
  color: #64748b;
  text-transform: uppercase;
  font-size: 7pt;
  margin-bottom: 2px;
}
.meta-item span {
  font-weight: 700;
  color: #0f172a;
}
h2 {
  font-size: 12pt;
  font-weight: 700;
  color: #0f172a;
  border-left: 3.5px solid #2563eb;
  padding-left: 8px;
  margin: 22px 0 10px 0;
  text-transform: uppercase;
  letter-spacing: 0.2px;
  page-break-after: avoid;
}
h3 {
  font-size: 10pt;
  font-weight: 700;
  color: #1e293b;
  margin: 14px 0 6px 0;
  page-break-after: avoid;
```

```
  }
  p, ul, ol {
    margin: 0 0 10px 0;
  }
  ul, ol {
    padding-left: 20px;
  }
  li {
    margin-bottom: 4px;
  }
  pre {
    background-color: #0f172a;
    color: #e2e8f0;
    padding: 12px 14px;
    border-radius: 5px;
    font-family: "SFMono-Regular", Consolas, "Liberation Mono", Menlo, Courier,
monospace;
    font-size: 7.5pt;
    line-height: 1.35;
    overflow-x: auto;
    white-space: pre-wrap;
    word-break: break-all;
    border: 1px solid #1e293b;
    page-break-inside: avoid;
    margin: 8px 0 16px 0;
  }
  .table-container {
    width: 100%;
```

```
  margin: 10px 0 16px 0;
  page-break-inside: avoid;
}
table {
  width: 100%;
  border-collapse: collapse;
  font-size: 8pt;
}
th, td {
  border: 1px solid #cbd5e1;
  padding: 6px 8px;
  text-align: left;
}
th {
  background-color: #f1f5f9;
  color: #0f172a;
  font-weight: 700;
  text-transform: uppercase;
  font-size: 7.5pt;
}
.badge {
  display: inline-block;
  padding: 2px 6px;
  border-radius: 4px;
  font-size: 7pt;
  font-weight: 700;
  text-transform: uppercase;
}
```

```
    .badge-blue { background-color: #dbeafe; color: #1e40af; }
    .badge-amber { background-color: #fef3c7; color: #92400e; }
    .badge-green { background-color: #dcfce7; color: #166534; }
    .page-break {
      page-break-before: always;
    }
  </style>
</head>
<body>

  <div class="header-block">
    <h1 class="header-title">PumpOS &bull; System Specification & Source Blueprint</h1>
    <p class="header-subtitle">Automated Petrol Pump Accounting, Meter Reconciliation & Secure Backup Engine</p>
  </div>

  <div class="meta-grid">
    <div class="meta-item">
      <strong>Architecture Standard</strong>
      <span>Next.js 14 / TypeScript</span>
    </div>
    <div class="meta-item">
      <strong>Relational Storage</strong>
      <span>PostgreSQL + Prisma ORM</span>
    </div>
    <div class="meta-item">
      <strong>Disaster Recovery</strong>
      <span>AES-256 Cloud Object Backup</span>
```

```
  </div>
  <div class="meta-item">
    <strong>Reconciliation Target</strong>
    <span>Zero-Discrepancy Shift Engine</span>
  </div>
</div>

<h2>1. Executive Summary & Design Principles</h2>
<p>PumpOS is designed to manage end-to-end retail fuel operations. The system eliminates manual shift calculation errors, automates stock deduction based on nozzle meter delta values, settles multi-channel payment streams (Cash, POS, QR/UPI, Fleet Ledger), and streams encrypted database snapshots to object storage.</p>

<ul>
  <li><strong>Error Prevention (Poka-Yoke):</strong> Meter rollbacks and negative volumes are rejected prior to database writes.</li>
  <li><strong>Zero-Overhead Local State:</strong> Client-side reactive math provides instant discrepancy flags without server roundtrips.</li>
  <li><strong>Immutable Auditability:</strong> Shift sign-offs, manager overrides, and meter calibrations generate non-repudiable audit logs.</li>
</ul>

<h2>2. Project Structure</h2>
<pre>petrol-pump-os/
├── .env.example
├── package.json
├── tsconfig.json
├── prisma/
|   ├── schema.prisma
|   └── seed.ts
```

├── lib/

|  ├── prisma.ts

|  └── notifications.ts

├── services/

|  └── shiftReconciliation.service.ts

├── scripts/

|  ├── backup-db.ts

|  └── restore-db.ts

└── app/

  ├── layout.tsx

  ├── globals.css

  ├── page.tsx

  └── api/shifts/close/route.ts</pre>

<h2>3. Environment Configuration (.env.example)</h2>

<pre>DATABASE_URL="postgresql://postgres:password@db.xxxx.supabase.co:5432/postgres?sslmode=require"

BACKUP_ENCRYPTION_KEY="0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"

S3_REGION="us-east-1"

S3_BUCKET_NAME="station-backups"

S3_ACCESS_KEY_ID="YOUR_ACCESS_KEY_ID"

S3_SECRET_ACCESS_KEY="YOUR_SECRET_ACCESS_KEY"

TWILIO_ACCOUNT_SID="ACXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"

TWILIO_AUTH_TOKEN="your_auth_token"

TWILIO_PHONE_NUMBER="+1234567890"

MANAGER_ALERT_PHONE="+919876543210"</pre>

<h2>4. Production Database Schema (prisma/schema.prisma)</h2>

<pre>datasource db {

provider = "postgresql"

url = env("DATABASE_URL")

}

generator client {

provider = "prisma-client-js"

}

enum Role { SUPER_ADMIN; MANAGER; OPERATOR; ACCOUNTANT }

enum FuelType { PETROL_91; PETROL_95; DIESEL; PREMIUM_DIESEL; CNG; EV_FAST }

enum ShiftStatus { OPEN; PENDING_RECONCILIATION; CLOSED; DISCREPANCY_FLAGGED }

enum PaymentMode { CASH; POS_CARD; UPI_QR; FLEET_CREDIT; VOUCHER;
BANK_TRANSFER }

enum LedgerEntryType { DEBIT; CREDIT }

model User {

id String @id @default(uuid())

email String @unique

phone String @unique

passwordHash String

name String

role Role @default(OPERATOR)

isActive Boolean @default(true)

```
  createdAt    DateTime  @default(now())
  updatedAt    DateTime  @updatedAt
  assignedShifts Shift[] @relation("ShiftAttendant")
  approvedShifts Shift[] @relation("ShiftManager")
  meterReadings MeterReading[]
  auditLogs     AuditLog[]
}

model Tank {
  id           String    @id @default(uuid())
  tankNumber     String    @unique
  fuelType       FuelType
  capacityLitres Decimal   @db.Decimal(10, 2)
  currentStock   Decimal   @db.Decimal(10, 2)
  deadStockLevel Decimal   @default(500.00) @db.Decimal(10, 2)
  dispensers     Dispenser[]
  dipReadings    TankDipReading[]
}

model Dispenser {
  id        String    @id @default(uuid())
  pumpNumber String    @unique
  tankId     String
  tank       Tank      @relation(fields: [tankId], references: [id])
  nozzles    Nozzle[]
}

model Nozzle {
```

```
  id          String        @id @default(uuid())
  nozzleNumber Int
  dispenserId  String
  fuelType     FuelType
  lastReading  Decimal       @default(0.00) @db.Decimal(12, 2)
  dispenser    Dispenser     @relation(fields: [dispenserId], references: [id], onDelete:
Cascade)
  readings     MeterReading[]
  @@unique([dispenserId, nozzleNumber])
}

model Shift {
  id           String        @id @default(uuid())
  shiftNumber    Int
  status         ShiftStatus    @default(OPEN)
  openedAt       DateTime       @default(now())
  closedAt       DateTime?
  attendantId    String
  managerId      String?
  totalFuelSales Decimal       @default(0.00) @db.Decimal(12, 2)
  cashExpected   Decimal       @default(0.00) @db.Decimal(12, 2)
  cashCollected  Decimal       @default(0.00) @db.Decimal(12, 2)
  cashShortage   Decimal       @default(0.00) @db.Decimal(10, 2)
  attendant      User          @relation("ShiftAttendant", fields: [attendantId], references: [id])
  manager        User?         @relation("ShiftManager", fields: [managerId], references: [id])
  meterReadings  MeterReading[]
  sales          Sale[]
}
```

```
model MeterReading {
  id            String   @id @default(uuid())
  shiftId       String
  nozzleId      String
  operatorId    String
  openingReading Decimal  @db.Decimal(12, 2)
  closingReading Decimal  @db.Decimal(12, 2)
  testingVolume  Decimal  @default(0.00) @db.Decimal(6, 2)
  netSoldVolume  Decimal  @db.Decimal(10, 2)
  unitPrice     Decimal  @db.Decimal(10, 2)
  totalAmount   Decimal  @db.Decimal(12, 2)
  shift         Shift    @relation(fields: [shiftId], references: [id], onDelete: Cascade)
  nozzle        Nozzle   @relation(fields: [nozzleId], references: [id])
  operator      User     @relation(fields: [operatorId], references: [id])
}

model Sale {
  id          String     @id @default(uuid())
  shiftId     String
  paymentMode PaymentMode
  amount      Decimal    @db.Decimal(12, 2)
  customerId  String?
  shift       Shift      @relation(fields: [shiftId], references: [id], onDelete: Cascade)
  customer    Customer?  @relation(fields: [customerId], references: [id])
}

model Customer {
```

```
  id            String          @id @default(uuid())
  accountName   String
  phone         String          @unique
  creditLimit   Decimal         @default(0.00) @db.Decimal(12, 2)
  currentBalance Decimal        @default(0.00) @db.Decimal(12, 2)
  sales         Sale[]
  ledger        CustomerLedger[]
}

model CustomerLedger {
  id          String         @id @default(uuid())
  customerId  String
  entryType   LedgerEntryType
  amount      Decimal        @db.Decimal(12, 2)
  balanceAfter Decimal       @db.Decimal(12, 2)
  description String
  customer    Customer       @relation(fields: [customerId], references: [id])
}

model AuditLog {
  id       String   @id @default(uuid())
  userId   String
  action   String
  resource String
  recordId String?
  newValue Json?
  createdAt DateTime @default(now())
  user     User     @relation(fields: [userId], references: [id])
```

}</pre>

<h2>5. Shift Reconciliation Engine (services/shiftReconciliation.service.ts)</h2>

<pre>import { PrismaClient, ShiftStatus, PaymentMode, LedgerEntryType } from "@prisma/client";
import { Decimal } from "@prisma/client/runtime/library";

export class ShiftReconciliationService {
  public static async reconcileAndCloseShift(prisma: PrismaClient, payload: any) {
    const { shiftId, managerId, nozzleReadings, payments, cashCollected } = payload;
    const actualCash = new Decimal(cashCollected);

    return await prisma.$transaction(async (tx) => {
      const shift = await tx.shift.findUnique({ where: { id: shiftId } });
      if (!shift || shift.status !== ShiftStatus.OPEN) throw new Error("Invalid Shift");

      let totalGrossRevenue = new Decimal(0);
      let totalVolumeSold = new Decimal(0);

      for (const item of nozzleReadings) {
        const nozzle = await tx.nozzle.findUnique({
          where: { id: item.nozzleId },
          include: { dispenser: true }
        });
        if (!nozzle) throw new Error("Nozzle not found");

```
  const opening = nozzle.lastReading;
  const closing = new Decimal(item.closingReading);
  const testing = new Decimal(item.testingVolume || 0);

  if (closing.lessThan(opening)) throw new Error("Meter rollback detected");
  const netSoldVolume = closing.minus(opening).minus(testing);

  const fuelPrice = await tx.fuelPrice.findUnique({ where: { fuelType: nozzle.fuelType } });
  const lineAmount = netSoldVolume.times(fuelPrice!.sellingPrice);

  totalGrossRevenue = totalGrossRevenue.plus(lineAmount);
  totalVolumeSold = totalVolumeSold.plus(netSoldVolume);

  await tx.meterReading.create({
    data: {
      shiftId, nozzleId: nozzle.id, operatorId: shift.attendantId,
      openingReading: opening, closingReading: closing, testingVolume: testing,
      netSoldVolume, unitPrice: fuelPrice!.sellingPrice, totalAmount: lineAmount,
    }
  });

  await tx.nozzle.update({ where: { id: nozzle.id }, data: { lastReading: closing } });
  await tx.tank.update({
    where: { id: nozzle.dispenser.tankId },
    data: { currentStock: { decrement: netSoldVolume } }
  });
}
```

```
    let digitalCollections = new Decimal(0);
    let fleetCreditTotal = new Decimal(0);

    for (const p of payments) {
      const amount = new Decimal(p.amount);
      if (p.paymentMode === PaymentMode.FLEET_CREDIT) {
        const customer = await tx.customer.findUnique({ where: { id: p.customerId } });
        const newBal = customer!.currentBalance.plus(amount);
        await tx.customer.update({ where: { id: customer!.id }, data: { currentBalance: newBal }
});
        await tx.customerLedger.create({
          data: { customerId: customer!.id, entryType: LedgerEntryType.DEBIT, amount,
balanceAfter: newBal, description: "Fuel Credit" }
        });
        fleetCreditTotal = fleetCreditTotal.plus(amount);
      } else if (p.paymentMode !== PaymentMode.CASH) {
        digitalCollections = digitalCollections.plus(amount);
      }
    }

    const expectedCash =
totalGrossRevenue.minus(digitalCollections).minus(fleetCreditTotal);
    const cashDiscrepancy = actualCash.minus(expectedCash);
    const hasDiscrepancy = cashDiscrepancy.abs().greaterThan(new Decimal(5.0));
    const finalStatus = hasDiscrepancy ? ShiftStatus.DISCREPANCY_FLAGGED :
ShiftStatus.CLOSED;

    await tx.shift.update({
      where: { id: shiftId },
```

```
      data: { managerId, status: finalStatus, closedAt: new Date(), totalFuelSales: totalGrossRevenue, cashExpected: expectedCash, cashCollected: actualCash, cashShortage: cashDiscrepancy.isNegative() ? cashDiscrepancy.abs() : 0 }
    });

    return { shiftId, totalGrossRevenue: totalGrossRevenue.toString(), cashDiscrepancy: cashDiscrepancy.toString(), status: finalStatus };
  });
 }
}</pre>

 <div class="page-break"></div>

 <h2>6. Encrypted Disaster Recovery Pipeline (scripts/backup-db.ts)</h2>
 <pre>import { spawn } from "child_process";
import { createCipheriv, randomBytes, scryptSync } from "crypto";
import { PassThrough } from "stream";
import { S3Client } from "@aws-sdk/client-s3";
import { Upload } from "@aws-sdk/lib-storage";

async function runEncryptedBackup() {
 const { DATABASE_URL, BACKUP_ENCRYPTION_KEY, S3_BUCKET_NAME, S3_REGION, S3_ACCESS_KEY_ID, S3_SECRET_ACCESS_KEY } = process.env;
 const timestamp = new Date().toISOString().replace(/[:.]/g, "-");
 const s3ObjectKey = `backups/${timestamp}_pump_backup.sql.gz.enc`;

 const s3Client = new S3Client({
  region: S3_REGION || "us-east-1",
```

```
    credentials: { accessKeyId: S3_ACCESS_KEY_ID!, secretAccessKey:
S3_SECRET_ACCESS_KEY! },
  });

  const salt = randomBytes(16);
  const derivedKey = scryptSync(BACKUP_ENCRYPTION_KEY!, salt, 32);
  const iv = randomBytes(16);
  const cipher = createCipheriv("aes-256-cbc", derivedKey, iv);

  const pgDump = spawn("pg_dump", ["--dbname", DATABASE_URL!, "--format=plain", "--
compress=6"]);
  const uploadStream = new PassThrough();
  uploadStream.write(salt);
  uploadStream.write(iv);

  pgDump.stdout.pipe(cipher).pipe(uploadStream);

  const parallelUpload = new Upload({
    client: s3Client,
    params: { Bucket: S3_BUCKET_NAME, Key: s3ObjectKey, Body: uploadStream,
ContentType: "application/octet-stream" },
  });

  await parallelUpload.done();
  console.log(`Backup completed: ${s3ObjectKey}`);
}
runEncryptedBackup();</pre>

  <h2>7. Production Deployment & Operational Runbook</h2>
```

```
<div class="table-container">
  <table>
    <thead>
      <tr>
        <th>Stage</th>
        <th>Command / Action</th>
        <th>Verification Target</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td><strong>1. Database Provisioning</strong></td>
        <td>Create PostgreSQL instance on Supabase / Neon</td>
        <td>Direct connection URI available</td>
      </tr>
      <tr>
        <td><strong>2. Schema Sync</strong></td>
        <td><code>npx prisma db push</code></td>
        <td>All 11 relational tables & enums provisioned</td>
      </tr>
      <tr>
        <td><strong>3. Station Baseline</strong></td>
        <td><code>npm run db:seed</code></td>
        <td>Pumps, pricing matrix, and tanks loaded</td>
      </tr>
      <tr>
        <td><strong>4. Shift Verification</strong></td>
        <td><code>npm run dev</code> &bull; Enter readings & sign off</td>
```

```
        <td>Cash variance audit completes inside 5ms</td>
      </tr>
      <tr>
        <td><strong>5. Automated Backup</strong></td>
        <td>Crontab: <code>59 23 * * * npm run backup</code></td>
        <td>Encrypted stream uploaded directly to S3</td>
      </tr>
    </tbody>
  </table>
 </div>

</body>
</html>
```